AOSP 进阶丛书 · 第七册 · ANDROID 13 (API 33)

系统工程师面试题集

82 道系统级面试题：Binder、init/Zygote 启动、Input 事件分发、AMS/WMS、ROM · BSP · 刷机与综合场景——每题三件套：问题、带源码锚点的深度解答、面试官追问链。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：把六册读成你的话术 2
第一章 Binder（15 题） 2
第二章 启动主线：init / Zygote（15 题） 5
第三章 Input 事件分发（10 题） 9
第四章 AMS 与进程管理（12 题） 11
第五章 WMS 与渲染（10 题） 14
第六章 ROM / BSP / 刷机（12 题） 16
第七章 综合场景题（8 题） 18
第八章 深水区题（20 题） 20
第九章 补全题（10 题） 25
附录 面试策略五条 27

本册速览

112 题九族：Binder 15 · 启动 15 · Input 10 · AMS 12 · WMS 10 · ROM/BSP 12 · 综合 8 · 深水区 20 · 补全 10
每题三件套：问题 + 带源码锚点的深度解答 + 追问链
每题标注〔册·章〕可回查六册正文
用法：先自答再看解答；追问链务必脑内走一遍
面试前夜：重读各册设计决策章 + 本册附录策略五条

前言：把六册读成你的话术

本册是前六册的"输出化训练"：**112 道系统级面试题，每题三件套——问题、深度解答（带源码锚点）、追问链（面试官的下一刀）**。用法建议：先自答再看解答；追问链务必在脑内走一遍——面试的差距不在第一问，在第三问。

解答里的行号与前六册一致（本树 Android 13 实测）。标注〔册·章〕表示出处，可回查。

第一章 Binder（15 题）

Q1.1 Binder 为什么只拷贝一次数据?

【深度解答】接收进程对 /dev/binder mmap 一块 1MB-2 页只读区（ProcessState.cpp:47/503），内核把该区同时映射进内核地址空间。发送时驱动的 binder_transaction 里一次 copy_from_user 直写这块双重映射区，接收方读自己的映射即得数据，省掉 copy_to_user。配套设计：PROT_READ 最小权限、MAP_NORESERVE 省内存、MAP_PRIVATE 隔离回写。对照：管道两次拷贝、共享内存零次但无安全/同步语义——Binder 是"每微秒买最多语义"。〔册3·一〕

【追问链】→ 为什么不全部用共享内存？→ Binder 的能力传递（flat_binder_object 翻译句柄）怎么做？→ AIDL 传大位图走什么路径（fd+共享内存）？

Q1.2 Binder 的安全模型为什么可信?

【深度解答】身份在内核采集：binder_transaction 把 current->cred 的 euid/pid 写进事务（sender_euid），服务端 executeCommand 装载为 mCallingUid/mCallingPid（IPCThreadState.cpp:1269-1273），任何用户态伪造无效。第二层是 SELinux 上下文随事务携带（BR_TRANSACTION_SEC_CTX，:1227）。第三层是 servicemanager 名录准入（canAdd/canFind）与 VINTF 声明一致性。三层职责：名录管"找得到"，驱动管"你是谁"，服务管"你能干什么"。〔册3·五/七〕

【追问链】→ clearCallingIdentity 为什么存在、忘恢复会怎样？→ 普通 App 能伪造 getCallingUid 吗？→ SELinux 拦截发生在哪一层？

Q1.3 一次同步 Binder 调用的完整十跳?

【深度解答】①getService（ActivityManager.java:4675，Singleton 缓存）②Proxy.transact→BinderProxy.transactNative(:614) ③JNI android_os_BinderProxy_transact(:1387) ④IPCThreadState.transact(:706)→writeTransactionData(BC_TRANSACTION :725) ⑤talkWithDriver ioctl(:997) ⑥驱动 binder_transaction：验句柄/采身份/一次拷贝/入队唤醒 ⑦服务端 executeCommand(:1147) 收 BR_TRANSACTION_SEC_CTX ⑧JavaBBinder→Binder.execTransact→Stub.onTransact ⑨业务执行 BC_REPLY ⑩客户端 BR_REPLY(:939) 零拷贝装 reply 返回。〔册3·九〕

【追问链】→ oneway 在哪一跳分岔（flags 一个比特）？→ 嵌套调用（服务端再调别人）如何不错乱（transaction_stack）？→ 十跳哪一跳最容易成为性能瓶颈（⑦调度唤醒）？

Q1.4 oneway 的真实语义三个误区?

【深度解答】误区一"立即执行"：oneway 到 BR_TRANSACTION_COMPLETE（:921）即返回，只保证"内核已收件"；误区二"天然并发"：同一目标的 oneway 在驱动内单队列保序（async_todo），跨进程才可能乱；误区三"随便发"：async 空间约为映射区一半，塞满即 BR_FAILED_REPLY——大包 oneway 翻车的根源。Android 13 另有泛洪检测 BR_ONEWAY_SPAM_SUSPECT(:915)。〔册3・三/十〕

【追问链】→ 心跳类 oneway 该怎么设计（分包/降频）？→ oneway 大数据的替代方案？→ AIDL 里哪些方法天然该 oneway（回调/通知类）？

Q1.5 Binder 线程池 15+1 的供给制怎么运转?

【深度解答】主线程由 startThreadPool 进入（App 在 zygoteInit→onZygoteInit:92-96），BC_ENTER_LOOPER；驱动按并发压力发 BR_SPAWN_LOOPER(:1373)→spawnPooledThread→BC_REGISTER_LOOPER，上限 BINDER_SET_MAX_THREADS（普通 15，system_server 31，SystemServer.java:435/854）。线程退出 BC_EXIT_LOOPER。排障铁律：线程打满+全忙=新同步调用排队（表现为对端"卡"而非自身卡）。〔册3・十〕

【追问链】→ 为什么 system_server 翻倍？→ binder 线程和 Handler 线程的关系（事务到 ApplicationThread 再转主线程 Looper）？→ ANR 时 binder 栈长什么样（debuggerd 三类栈型）？

Q1.6 servicemanager 为什么零线程池、仅 oneway?

【深度解答】名录是全机寻址根，必须极稳极简：setThreadPoolMaxThreadCount(0)（main.cpp:125）杜绝任何客户端把名录挂住；FATAL_IF_NOT_ONEWAY(:126) 从机制上禁同步调用；Looper 单线程 pollAll(-1) 天然串行化注册与查询。自举：addService("manager", this)(:129)+becomeContextManager(:134) 拿下句柄 0。〔册3・七〕

【追问链】→ 句柄 0 为什么不能给别的服务？→ servicemanager 死了会怎样（critical 连带重启整类服务）？→ 三域（binder/hwbinder/vndbinder）为什么隔离？

Q1.7 注册服务时的 linkToDeath 有什么用?

【深度解答】addService 的五道关卡之一（ServiceManager.cpp:325）：注册即挂死亡通知，服务进程死→驱动向 ref 广播 BR_DEAD_BINDER→servicemanager 自动除名，名录永无"僵尸服务"。同一机制的著名客户：AMS 的 AppDeathRecipient（ProcessList.java:5056）监 App 进程死、清理组件与窗口。〔册3・七/十〕

【追问链】→ linkToDeath 在客户端怎么用（DeathRecipient+unlink 必须成对）？→ BinderProxy 泄漏与死亡通知的关系（gNumProxies 告警）？→ system_server 重启后 App 侧如何自愈（缓存失效重取）？

Q1.8 Parcel 为什么是"事务即域"?

【深度解答】服务端事务里的 data/reply 由 binder 映射区背书，事务返回缓冲即被回收（freeBuffer）。跨事务保存 Parcel=持有将被复用的内核内存，内容随时被下一事务覆盖。三类易错：异步线程延迟读、缓存 Parcel、子线程复用——症状都是"数据偶尔变花"。〔册3 · 六〕

【追问链】→ 要跨事务传大数据怎么办（fd/共享内存/分片）？→ writeStrongBinder 传的到底是什么（flat_binder_object 二态）？→ aidl 的 in/out/inout 与 Parcel 生命周期关系?

Q1.9 BinderProxy 与 BpBinder 为什么要两层?

【深度解答】native BpBinder 持 32 位句柄、走 mIn/mOut 命令流；Java BinderProxy 只是它的马甲（javaObjectForIBinder:749 建 BinderProxyNativeData 持 BpBinder，每 IBinder 每进程一个代理，gNumProxies 计数告警防泄漏）。好处：Java GC 世界与 native 生命周期解耦（DeathRecipientList 缝合）、JNI 边界窄化为指针存取。〔册3 · 八〕

【追问链】→ 同一服务取两次 getService 返回同一对象吗（缓存+getInstance 复用）？→ Binder 本地对象跨进程传回自己进程会怎样（javaObjectForIBinder 直返原对象）？→ 为什么 Java 侧不直接操作 ioctl?

Q1.10 AIDL 生成的 Stub/Proxy 结构与协议保护?

【深度解答】Stub.onTransact 按 code 分发（TRANSACTION_x=FIRST_CALL_TRANSACTION +n），入口 enforceInterface 校验 DESCRIPTOR；Proxy 持 mRemote（BinderProxy），写 token→参数→transact→读 reply。手写骨架三十行（册3 §8.3）覆盖 90% 场景；稳定 AIDL 由版本目录（aidl_api/1,2…）冻结，跨分区接口按 Stability/VINTF 等级约束演化。〔册3 · 八〕

【追问链】→ in/out 参数在生成代码里的差异？→ 两个进程 DEscriptor 不一致会怎样（SecurityException/坏包）？→ rust/ndk backend 与 java backend 互通吗（同一接口多后端）？

Q1.11 冻结进程的 Binder 行为（12+）?

【深度解答】缓存 App 被 cgroup freezer 冻结：CPU 全停、发往它的同步事务立即 BR_FROZEN_REPLY（waitForResponse:931）快速失败而非挂起超时；oneway 与广播默认 deferred（解冻补投）。设计动机：把"冻结"从模糊语义变成协议级确定行为，后台冻结才敢默认开。〔册3 · 十〕

【追问链】→ 冻结时它发出的调用呢？→ 谁冻结谁解冻（OomAdjuster→cgroup；AMS 解冻窗口期）？→ FROZEN_REPLY 业务侧如何兜底（重试/退避）？

Q1.12 嵌套 Binder 调用如何支持"递归"?

【深度解答】每个 binder 线程有 transaction_stack：A 调 B，A 线程压栈等待；B 处理中调 C，B 的服务线程以调用方身份压栈；C 回 B、B 回 A，逐层弹栈——调用链跨三个进程而不乱，靠的是"栈随线程走"。这也是 binder 线程"双面性"（能服务也能外呼）的基础。〔册3 · 三〕

【追问链】→ A 等 B、B 反过来等 A 会怎样（死锁，system_server 为何禁同步外呼）？→ 服务端如何知道自己被嵌套调用（getCallingUid 即上跳身份）？→ oneway 有栈吗（无，async 队列）？

Q1.13 一次 Binder 调用大约多贵?

【深度解答】十跳耗时账本：组包微秒级、ioctl 1-2μs、驱动 5-20μs（含一次 copy）、目标唤醒受调度影响（数 μs~ms）、业务自定、回程对称。同机往返常识值 0.1-0.5ms。性能问题九成在唤醒调度与业务，协议本身极少瓶颈——perfetto binder 轨道的正确读法。〔册3・九〕

【追问链】→ 如何测量一次调用的真实开销（systrace 埋点/binder_calls_stats）？→ 批量化接口为什么是常见优化（一批一往返）？→ 缓存 BinderProxy 的收益边界?

Q1.14 Binder 驱动的核心数据结构关系?

【深度解答】五件套：binder_proc（进程，threads/nodes/refs 树）→ binder_thread（线程，transaction_stack）→ binder_node（服务端点，ptr/cookie 指宿主 BBinder）→ binder_ref（每进程侧句柄 desc）→ binder_buffer（接收区账本，async 半区独立）。用户态对照：node↔BBinder（JavaBBinder）、ref.desc↔BpBinder.mHandle、Parcel↔buffer 一段+offsets。〔册3・三〕

【追问链】→ App 拿到的"activity"到底是什么（system_server 里 AMS node 的 ref）？→ AMS 死了句柄怎么办（BR_DEAD_BINDER 广播）？→ 为什么 offsets 表与数据平面分离（对象槽扫描）？

Q1.15 hwbinder/vndbinder 与 binder 为什么三域?

【深度解答】Treble 把框架/硬件/厂商通信隔离成三个 binder 设备与三套 servicemanager：框架演进不污染 vendor 进程、vendor 崩溃不波及框架、SELinux 分别治理。AIDL HAL（Android 10+）已把 HAL 拉回主域复用工具链，但三域仍是现状（书3 §7 图）。〔册3・七〕

【追问链】→ 为什么不干脆合并成一张名录（版本/信任边界不同）？→ AIDL HAL 走 binder 还是 hwbinder（主域）？→ 判断一个服务在哪域的最快方法（service list 分段/dumpsys -l）？

第二章 启动主线：init / Zygote（15 题）

Q2.1 init 为什么分两阶段、两次 re-exec?

【深度解答】鸡生蛋问题：SELinux 策略与 rc 剧本都在 /system 分区，而第一阶段只拥有 ramdisk——先 FirstStageMain 挂分区（first_stage_mount.cpp:87）+switch_root，再 SetupSelinux（selinux.cpp:938）装载策略，最后 SecondStageMain（init.cpp:824）干活。re-exec 的意义：每个阶段只信任"自己亲手布置好的世界"，安全状态切换天然彻底。〔册2・二〕

【追问链】→ ramdisk 里的 init 和 system 的 init 是同一个二进制吗（是，main.cpp 四身份）？→ 不分两阶段会怎样（无策略可加载/服务无标签）？→ GKI 后 ramdisk 为何拆成 init_boot 分区？

Q2.2 init 为什么坚持单线程?

【深度解答】三个词：无锁（全局零 mutex，时序可推理——:829-832 注释明言 signalfd 之前禁起线程）、公平（ExecuteOneCommand 每轮一条命令，:94）、确定（同 rc 同顺序，bootchart 可度量）。代价：单条命令阻塞卡住 init（exec 命令因此走子进程+管道同步补偿）。对照 system_server 的并发世界——init 只服务"顺序的世界"。〔册2 · 五〕

【追问链】→ SIGCHLD 为什么用 signalfd 而非信号处理器？→ 主循环为什么先 reap 再处理其他 fd（防 ctl.start 竞态 :1025-1028）？→ 属性等待（wait_for_prop）为什么暂停推进命令？

Q2.3 late-init 触发瀑布的顺序为什么不可调换？

【深度解答】init.rc:516-547 的子触发器顺序即依赖序：early-fs（vold 起步）→fs/post-fs（挂载）→post-fs-data（/data 解密、persist 属性、apexd）→zygote-start（加密状态三选一触发）→boot（class_start）。把 zygote-start 提前：zygote 预加载要读 /data 的 dex/资源，且 ro.crypto.state 未定无法选触发分支——启动直接 fatal 或反复重启。〔册2 · 四〕

【追问链】→ 充电模式为什么跳过这条瀑布（charger 触发器 :979）？→ 属性触发器与事件触发器怎么配合（zygote-start 三条件）？→ 新增守护进程的 rc 该放哪个阶段？

Q2.4 zygote 为什么禁止起线程？

【深度解答】fork 多线程进程只保证调用方线程存活，其余线程连同持有的锁状态一起消失——子进程里这些锁永远不可能释放（POSIX 明确）。startZygoteNoThreadCreation（ZygoteInit.java:826）从源头杜绝；USAP 补水等原生操作也在受控点进行。衍生命题：App 的多线程是从 fork 之后才开始的。〔册2 · 六〕

【追问链】→ zygote 里的 binder 线程池什么时候起（fork 后子进程 onZygoteInit）？→ 为什么 preload 后要 gcAndFinalize（减小 COW 复制基数）？→ WebView 预加载为什么在 zygote（进程间共享）？

Q2.5 fork 而不是 exec 的原因？

【深度解答】exec 会重建地址空间，预加载的类/资源/ART 状态全部作废；fork+写时复制让 200MB 级框架世界"加载一次、千次复用"——App 秒起与全机内存水位都靠它。forkSystemServer（:683-792）硬编码出生证明：uid/gid 1000、13 项能力位图、nice-name system_server。〔册2 · 六〕

【追问链】→ fork 后子进程怎么降权到目标 uid（forkAndSpecialize 特化）？→ system_server 死了为什么要连累 zygote（critical 组全有或全无）？→ USAP 与普通 fork 的取舍？

Q2.6 runSelectLoop 的孵化流程？

【深度解答】ZygoteServer.java:425：poll（服务 socket+会话+USAP 管道）→processCommand 解析参数（uid/gid/ABI/caps/rlimits/seInfo）→Zygote.forkAndSpecialize(:347)→native 侧降权/关 fd/贴标签→子进程 return command(:562) 退出循环→zygoteInit(:980) 三步（commonInit/nativeZygoteInit 起 binder 池/applicationInit）→ActivityThread.main。父进程继续 poll，永续孵化。〔册2 · 六〕

【追问链】→ 谁向 zygote 发请求（AMS 的 ZygoteProcess 写 socket）？ → USAP 池空了会怎样（退化为普通 fork）？ → 32 位 App 由谁孵（辅 zygote，ABI 路由）？

Q2.7 preloaded-classes 清单是怎么来的、能改吗?

【深度解答】AOSP 用 tools/preload 从真实开机 trace 挖掘（类加载计数超阈值入选），是"框架最内核工作集"的活文档；Go 版走 --enable-lazy-preload（:855）延迟加载。厂商可裁剪——每删一行省一点内存、慢一点首帧，是内存-流畅权衡的最前线。〔册2 · 六/九〕

【追问链】→ 清单里挑一个类说明为什么全局需要它？ → lazy 模式下第一次 fork 前补加载的代价？ → 怎么验证裁剪效果（bootchart+内存基线对比）？

Q2.8 SystemServer 为什么分 Bootstrap/Core/Other 三批?

【深度解答】依赖拓扑的显式化：Bootstrap 建地基（Installer→AMS 占位→PMS 扫包→…），Core 放轻统计，Other 大合唱（WMS/IMS…按 boot phase 交错）。批内顺序同样是依赖序（书2 service-deps 图）；phase（100→1000）是全局进度条，服务在 onBootPhase 挂钩就位，避免"查无此人"。〔册2 · 七〕

【追问链】→ PMS 为什么必须 Bootstrap（AMS/Home 都等它）？ → AMS 先生 PMS 后熟是什么意思（占位 vs systemReady :2712/:2801）？ → 加新服务该进哪一批发在哪个 phase 挂钩?

Q2.9 boot phase 的 550 和 600 分别放行什么?

【深度解答】550（PHASE_ACTIVITY_MANAGER_READY）：AMS 就绪，系统内部可开始跨服务联动；600（PHASE_THIRD_PARTY_APPS_CAN_START）：三方 App 允许启动——Home 点火在 600 前后（startHomeActivity 链，ATMS:5874）。1000 是 BOOT_COMPLETED 广播之后。分层的意义：把"系统可用"拆成可观测的阶梯。〔册2 · 七/八〕

【追问链】→ sys.boot_completed 与 phase 1000 谁先（属性先于广播尾巴）？ → 开机动画退出的条件链（finishBooting 门闩 :5075）？ → 怎么观测当前 phase（dumpsys 各服务 onBootPhase 日志）？

Q2.10 开机完成的"权威标志"是什么?

【深度解答】sys.boot_completed=1（AMS.java:5149，finishBooting 内）——只有 AMS 有"框架就绪（Home 可启动、广播可送达）"的完整视角。init.svc.* 只证明服务在跑，dev.bootcomplete 是内核视角补充。三方 App 判定开机完成监听 ACTION_BOOT_COMPLETED 广播（有序，末位）。〔册2 · 八〕

【追问链】→ 为什么不 init 来设这个属性？ → BOOT_COMPLETED 广播被 ANR 拖住会怎样（有序链尾部等待）？ → 开机耗时怎么分段测（am 开机后 boot_time_event / perfetto）？

Q2.11 属性系统为什么读快写慢、且能这么早可用?

【深度解答】读：全进程 mmap 只读共享区，零系统调用（纳秒级）；写：socket 到 init 属性服务（property_service.cpp:527 handle_property_set_fd），SO_PEERCRED 凭证+SELinux 上下文仲裁（HandlePropertySet:481→PropertySet:176：合法性/ctl.* 分流/persist 持久化）。早可用

的原因：不依赖 ART/binder——init 第二阶段即建（PropertyInit:887）。〔册1・六/十〕

【追问链】→ ctl.start 与 setprop 的关系（前缀劫持为控制命令）？→ persist.* 何时落盘（/data/property，post-fs-data 后可写）？→ ro.* 为什么只读（设后拒绝 PREADONLY）？

Q2.12 ueventd 和 init 的关系？

【深度解答】同一二进制的另一身份（main.cpp:61 argv[0] 切换）：监听内核 uevent、按 ueventd.rc 建 /dev 节点与权限、固件加载（firmware_handler）。主流程 wait_for_coldboot_done（init.cpp:960）等它冷插完成才继续——所以"ueventd 先跑、init 等、再触发"。〔册2・二〕

【追问链】→ /dev 里没有 udev 为什么也能工作？→ 设备节点权限错了找哪个 rc？→ fastboot/recovery 下 ueventd 也跑吗（recovery 有独立 rc）？

Q2.13 A/B 分区与 AVB 如何配合保证升级安全？

【深度解答】A/B：双槽位（slot A/B）无缝升级，boot_control HAL 切槽、update_engine 写后台槽；AVB（external/avb）：vbmeta 哈希链校验 boot/system/vendor…，回滚索引防降级，bootloader 按锁状态与校验结果亮橙/黄/红/绿屏。失败自动回退旧槽——"升级永不 brick"的两层保险。〔册1・七/notes・07〕

【追问链】→ 动态分区（super）解决什么（分区弹性）？→ 虚拟 A/B 的 COW 快照怎么做到不占双倍空间？→ 解锁后哪些校验仍在（仅 dm-verity 提示减弱）？

Q2.14 boot.img 里有什么、GKI 改了什么？

【深度解答】经典 boot.img=kernel+ramdisk（mkbootimg 头：页大小/基地址/标签）；Android 13 起 init_boot 分区装通用 ramdisk，boot 只剩 GKI 内核——通用内核镜像+设备 ramdisk 分离，内核碎片化治理的关键一步。dtbo 单独分区放设备树覆盖。〔册1・七/notes・07〕

【追问链】→ boot header v4 变了什么？→ vendor_boot 与 init_boot 的分工？→ fastboot flash boot 与 boot_a/boot_b 的关系（当前槽）？

Q2.15 从按下电源键到 Launcher，你能白板画出几层？

【深度解答】验收级答案分九段：BootROM→bootloader（AVB 校验）→kernel→init 一阶段（挂分区）→selinux_setup→init 二阶段（属性/epoll/解析 rc→触发瀑布）→zygote（preload→forkSystemServer→runSelectLoop）→SystemServer（三批服务+phase 推进+systemReady）→Home/BOOT_COMPLETED。每段至少一个锚点（dmesg"init second stage started!"、boot_progress_* 事件、sys.boot_completed）。〔册2・一〕

【追问链】→ 每一段的"完成信号"各是什么？→ 砍掉 zygote 预加载开机快多少、代价是什么？→ 面试官最爱追问的三个数字（线程 15+1/31、映射 1MB-2页、超时 5s/10s/20s）记牢了吗？

第三章 Input 事件分发（10 题）

Q3.1 一次触摸从指尖到 onTouchEvent 的完整链路?

【深度解答】七段：①内核 evdev（/dev/input/eventX）②EventHub.getEvents（EventHub.cpp:1533，epoll+inotify）③InputReader.loopOnce（:68→:103→:110）翻译成 NotifyMotionArgs ④InputDispatcher.notifyMotion(:3975) 入队→dispatchOnce(:602)→dispatchOnceInnerLocked(:718) 选目标 ⑤InputChannel socketpair（InputChannel.java:109 openInputChannelPair，addWindow 时配对注册）投给焦点窗口 ⑥App 侧 WindowInputEventReceiver→enqueueInputEvent→InputStage 责任链（Native→PreIme→Ime→View→ViewRootImpl）⑦View.dispatchTouchEvent（onTouchListener→onTouchEvent）。〔册5・五〕

【追问链】→ 事件为什么必须经过 system_server（仲裁/拦截/ANR 计时）？→ 通道为什么是 socketpair 不是 binder（高频小包+fd 复用+不被线程池策略绑架）？→ MOVE 的批量合并（resample）在哪层做（Reader/Dispatcher）？

Q3.2 输入 ANR 的 5 秒从哪一刻开始计?

【深度解答】从 dispatcher publishMotion（写入目标 socket）时刻起，AnrTracker 按截止期排堆；App 消费后 finishInputEvent 回执摘除。5s 未回执→InputManagerService.notifyANR→AMS AnrHelper.appNotResponding（:71，队列化执行 dump）→traces+弹窗。所以输入 ANR=主线程没能在 5s 内把事件"走完责任链并回执"。〔册5・五/六〕

【追问链】→ ANR 弹窗期间事件还投递吗（暂停该窗口投递）？→ 怎么从 traces 定位"谁堵了主线程"（看主线程栈+InputStage 位置）？→ 广播 10s/服务 20s 与输入 5s 的差异逻辑（在场感）？

Q3.3 View 事件分发的四条规则?

【深度解答】①拦截权在父（onInterceptTouchEvent，仅 ViewGroup）；②命中权在子（DOWN 逆 z 序 hit test）；③锁定起于 DOWN（mFirstTouchTarget 定终身，后续 MOVE/UP 不再命中测试）；④消费可中途易主（每轮 MOVE 复议拦截；子可 requestDisallowInterceptTouchEvent 投票但 DOWN 前无效）。滑动冲突官方解（内外拦截/投票）皆由此推导。〔册5・五〕

【追问链】→ ACTION_CANCEL 什么时候来（窗口失焦/被拦），收到必须做什么（清按下态）？→ 多指 split-touch 如何分发（可拆多窗口）？→ onTouchListener 与 onTouchEvent 谁先（前者，return true 短路）？

Q3.4 键盘事件与触摸事件的寻址差异?

【深度解答】触摸按"命中"（可见区域测试，可多目标 split）；按键按"焦点"（InputDispatcher 的 focusedWindow 唯一），路径先过系统拦截器（电源/音量短路）→窗口→ViewRootImpl.processKeyEvent→Ime 截胡→dispatchKeyEvent 自焦点 View 冒泡至 Activity。

两条寻址线在 WMS 汇合（焦点授予随窗口层级）。〔册5・六〕

【追问链】→ 焦点窗口为空时按键去哪（电话/全局策略）？→ 输入法为什么能截胡（ImeInputStage 位置）？→ back 键的手势导航（从区域命中转为 key 合成）？

Q3.5 dumpsys input 能回答哪些问题?

【深度解答】五段活地图：设备清单与配置（触摸屏分辨率/方向）、InputChannel 注册表（哪个窗口哪个 fd）、焦点窗口与触摸目标（寻址现场）、事件队列深度（积压=慢消费）、ANR 计时（哪些事件在等谁）。排障第一站：先看焦点/通道，再看队列，最后下钻 App。〔册5・附录〕

【追问链】→ focusedWindow 为空怎么排查（token/层序/启动态）？→ 队列深但 App 不卡说明什么（回执路径断/monitor）？→ getevent 与 dumpsys input 的结论矛盾时信谁（分层定界）？

Q3.6 为什么 Choreographer 四车道固定 INPUT→ANIMATION→INSETS→TRAVERSAL?

【深度解答】同帧因果闭环：先消化这一帧的输入，再算动画目标值，然后 insets 协商，最后一次性测量布局绘制——顺序颠倒会把输入延迟整整一帧（体感跟手性下降）。常量定义 Choreographer.java:226/256。〔册5・四〕

【追问链】→ doFrame 晚到怎么补偿（跳帧计数+对齐帧间隔）？→ 动画为什么能不阻塞主线程（RenderThread 属性动画）？→ postCallback 的排队单位是什么（车道+VSync）？

Q3.7 同步屏障是什么、为什么 traversal 能插队?

【深度解答】scheduleTraversals（ViewRootImpl.java:2257）postSyncBarrier 让 Looper 队列跳过普通消息、只取异步消息；vsync 到达的 doFrame 消息是异步的，得以越过积压消息立即执行；doTraversal 里 removeSyncBarrier 恢复公平。帧的时效性 > 消息公平性，窗口期极短所以可接受。〔册5・三〕

【追问链】→ 忘拔屏障会怎样（普通消息全部饿死，ANR）？→ Handler 消息能设异步吗（setAsynchronous，系统专用居多）？→ 这与消息优先级（when）是一回事吗（不是，屏障是二分闸）？

Q3.8 三缓冲为什么是三?

【深度解答】双缓冲：App 画一块、SF 合一块，任一方晚点就互等（帧率腰斩）；三缓冲加一块"排队垫"，容忍一次抖动（App 晚一帧用户看到的仍是旧帧而非空等）；四缓冲纯增延迟与内存。BufferQueue 由生产者（App 的 Surface）与消费者（SF/Layer）协商槽位。〔册5・四〕

【追问链】→ 帧在哪个缓冲里被"看见"（SF acquire 后 present）？→ Overlay 直通为什么能绕过 GPU（层不混合）？→ FrameTimeline 怎么标掉帧责任（expected vs actual 四段分责）？

Q3.9 窗口焦点、触摸目标、输入通道三者的关系?

【深度解答】通道是"管道"（addWindow 时配对注册，无通道窗口收不到事件），焦点是"地址"（key 寻址），触摸目标是"收件人"（motion 命中，可 split 多窗口）。三者独立：可见无焦点窗口可收触摸（如悬浮按钮）但收不到按键；有通道无焦点的对话框同屏可点。〔册5・一/六〕

【追问链】→ 点击穿透的机制解释（hit test 未命中本窗）？→ 多屏焦点（per-Display 独立）？→ 无障碍服务会改事件流吗（AccessibilityInputFilter 在责任链内）？

Q3.10 InputReader/Dispatcher/EventHub 三线程为什么不能合并?

【深度解答】三道隔离舱：EventHub 面对设备 IO 抖动（epoll 等待、插拔风暴），Reader 面对翻译复杂度（配置/坐标/手势合成/重采样），Dispatcher 面对策略（焦点/拦截/ANR 计时）。合并后任一环节卡=整条链卡；分离后队列解耦，可观测（各自的队列深度即水位线）。线程边界=可靠性边界。〔册5・五〕

【追问链】→ 触摸重采样（预测补偿）发生在哪层（Reader，相邻事件插值）？→ Dispatcher 卡住的表现（全机输入失灵，dumpsys input 队列爆炸）？→ inotify 热插拔新触摸屏何时可用（Reader 下轮 scan）？

第四章 AMS 与进程管理（12 题）

Q4.1 冷启动十步，进程切换发生在哪几步?

【深度解答】三次跨进程往返：①-②App→system_server（startActivity/ATMS）；④-⑤zygote 孵化（AMS→zygote socket→fork）与 attachApplication（新进程→AMS 回报）；⑥-⑦AMS→新进程（ClientTransaction）。App 侧八步（handleLaunchActivity→performLaunchActivity:3546→onCreate→首帧）全在本进程。观测对照 am start -W 的 TotalTime 与 systrace 轨道分段。〔册4・一/十〕

【追问链】→ 热启动少了哪段（孵化+attach）？→ system_server 视角的"启动完成"信号（activityIdle 回执）？→ 首帧和 TotalTime 谁先谁后（首帧先，idle 后）？

Q4.2 ClientTransaction 事务模型解决什么、怎么执行?

【深度解答】Android 9 前几十个 scheduleXxx 独立 binder 调用乱序风险高；事务化后一次投递"callbacks+终态"（realStartActivityLocked:902 组装），App 侧 TransactionExecutor 先跑 callbacks 再 executeLifecycleState——cycleToPath 沿状态机补中间态（onCreate→RESUME 必经 onStart），路径由执行器推导。批量性、可推理性、可扩展性（新增 Item 即新增周期能力）。〔册4・二〕

【追问链】→ 旋转重建是哪些 Item 的组合（ConfigChange/Relaunch）？→ activityIdle 为什么是"还款日"回执？→ 双向事务（App→AMS 走 ActivityClientController）同一模型吗?

Q4.3 四大组件的"欠条-兑现"分别叫什么?

【深度解答】mPendingServices（Service，bringUpServiceLocked:1179 进程不在先孵化）、mPendingBroadcast（Broadcast，等接收者进程）、mLaunchingProviders（Provider，等宿主发布）、Activity 的 pending 事务队列。统一兑现点：attachApplicationLocked（新进程回报 IApplicationThread 后集中还款）。〔册4・一/三〕

【追问链】→ 为什么集中兑现（收件人就位是共同前提）？ → 兑现顺序有讲究吗（Provider→Broadcast→Service 按语义急迫度）？ → 孵化失败欠条怎么清（进程死亡回调清账）？

Q4.4 有序广播为什么会连锁 ANR？goAsync 救什么？

【深度解答】有序队列串行：前一 receiver 不 finishReceiver 后面全部等待；超时线 10s（前台）/60s（后台，BroadcastQueue.java:203）到点 broadcastTimeoutLocked→AnrHelper。goAsync 把 onReceive 的处理窗口挪到自己线程（PendingResult），但超时线不变——finish 必须限时调。ANR 后广播链跳过该 receiver 继续（单点故障不扩散）。〔册4・四〕

【追问链】→ 并行队列为什么不易 ANR（不等回执）？ → manifest 接收者为什么进有序队列（可截获语义要求串行）？ → 粘性广播的现代替代（onStartCommand/WorkManager 回执）？

Q4.5 ContentProvider 为什么是唯一同步阻塞的组件入口？

【深度解答】调用方语义上要数据（query 返回值），无法欠条化：getContentProviderImpl（ContentProviderHelper.java:158）持 AMS 大锁、宿主不在则孵化并 Object.wait，publishContentProviders 唤醒；兜底 10s 超时。数据面不走 binder 大包：CursorWindow 共享内存分页、openFile 传 fd。〔册4・五〕

【追问链】→ 首次 query 卡顿的常见原因（宿主冷启动 500ms+）？ → 为什么 provider 能抬高宿主进程优先级（incProviderCount:261 计入 computeOomAdj）？ → multiprocess 声明为什么极少用？

Q4.6 computeOomAdj 的因子顺序为什么重要？

【深度解答】核算短路式：从 instrumentation/顶层/可见/前台服务/广播/服务/上应用/缓存 provider 逐项判，**命中即定档**（OomAdjuster.java:421 入口）。顺序=优先级：TOP_APP 永远最前。输出双刻度：PROCESS_STATE 给框架消费（JobScheduler 配额、binder 限制）、oom_score_adj 给 lmkd。〔册4・六〕

【追问链】→ 保活为什么难（自抬优先级的口子都要资源背书）？ → 冻结与杀的先后（先冻结缓冲带，压力升级才 kill）？ → lmkd 怎么知道该杀谁（adj 位图+PSI，LMK_PROCPRIO 命令族）？

Q4.7 Service 三种返回值与复活退避？

【深度解答】START_NOT_STICKY/STICKY/REDELIVER_INTENT 是"复活意愿声明"；ActiveServices 用指数退避（restartDelay 族常量）防杀-启死循环。前台服务=通知背书+优先级抬升；13 的 FGS 通知延迟（离开 10s 后显示部分类型）只是弱化打扰不取消透明。〔册4・三〕

【追问链】→ startForegroundService 的 5s 大限（未调 startForeground 即异常）？ → bind 与 start 两本账（双清才销毁）？ → 后台 startService 熔断（IllegalStateException）怎么绕（FGS/JobScheduler，别绕）？

Q4.8 ANR 的五个触发源与统一出口？

【深度解答】输入 5s（AnrTracker）、前台服务 20s/后台 200s、广播 10s/60s、provider 10s、（+Watchdog 系统级）。统一出口 AnrHelper.appNotResponding（:71）——队列化执行（不在触

发线程做 dump 重活），产出 /data/anr traces+dropbox，弹窗经权限校验。〔册4・四/六〕

【追问链】→ traces 里先看什么（主线程栈+锁持有者）？→ ANR 弹窗为什么另一个进程弹（不信任将死进程）？→ 线上 ANR 归因怎么建（ANR 前最后 binder 事务）？

Q4.9 AMS 与 ATMS/WMS 的分家边界?

【深度解答】2019 拆分：am/ 管进程与组件调度（ProcessRecord），wm/ 管任务/窗口/焦点（ActivityRecord/WindowState）；桥是 WindowProcessController（WMS 持有的进程视图）与 IApplicationThread（AMS 向进程发事务的通道）。窗口可见性回写影响 AMS 优先级——两视图必须联动。〔册4・一/册5・一〕

【追问链】→ 一个 Activity 的两边账本（ActivityRecord↔ActivityClientRecord，token 为桥）？→ 谁决定"后台杀谁"（AMS），谁决定"画到哪"（WMS）？→ 双视图架构的面试一句话（调度与画面解耦）？

Q4.10 进程从 zygote 出生到"可被调度组件"要过几关?

【深度解答】五关：fork 特化（uid/gid/caps/seInfo）→zygoteInit 三步（binder 线程池在 nativeZygoteInit 就位）→attachApplication 报到（ProcessRecord 入库）→集中兑现欠条（四组件）→首次 computeOomAdj 定档。此后进程进入调度生态（adj/cgroup/冻结三件套待遇）。〔册4・一/六〕

【追问链】→ attach 前收到对它的调用怎么办（等/排队，mProcessList 发起方挂起）？→ 进程起来但 Application.onCreate 卡住的后果（一切组件欠条挂着）？→ 预热（空白进程池）的现代做法与限制?

Q4.11 dumpsys activity 能拆出哪四本账?

【深度解答】activities（任务栈/ActivityRecord 状态机）、services（ServiceRecord/绑定/前台）、broadcasts（双队列与超时）、providers（发布表与引用计数）；外加 processes（ProcessRecord/adj/state）——第五本进程账。排障按"症状选账本"：卡启动看 activities，卡后台看 processes+lmkd。〔册4・附录〕

【追问链】→ 一个进程多条 adj 变化的时间线怎么查（events log am_adj）？→ broadcasts 队列里 receiver 卡住叫什么（dispatched 状态超时）？→ providers 引用计数泄漏的表现（宿主杀不掉）?

Q4.12 面试压轴：设计一个"保活"正确打开方式?

【深度解答】正面回答分三层：①合法驻留=前台服务（类型化通知）+ WorkManager 周期任务；②复活链路=STICKY/REDELIVER+JobScheduler 重试+BOOT_COMPLETED 补签；③接受现代现实：12+ 冻结与缓存池容量限制下，"保活"应改为"快速恢复"（进程死亡状态持久化+冷启动优化 <400ms）。把"对抗系统"翻译成"配合系统"才是加分答案。〔册4・六〕

【追问链】→ 冻结期间 FGS 会怎样（解冻窗口期）？→ 厂商白名单机制与 AOSP 差异（后台栈策略厂化）？→ 如何度量"恢复速度"（am start -W 分段+埋点）？

第五章 WMS 与渲染（10 题）

Q5.1 WMS/SF/App 的职责三分离?

【深度解答】WMS 管户口与层位（WindowState/token/z-order，无像素）；App 生产内容（View 树→Surface）；SF 合成上屏（跨进程 BufferQueue）。"系统从不碰你的像素，只调度你的画布"——崩溃域与性能域同时隔离：App 渲染崩不连累系统，SF 卡不拖垮窗口管理。〔册5・二〕

【追问链】→ SurfaceControl 与 Surface 的关系（图层句柄 vs 生产者接口）？→ relayoutWindow(:2219) 交付什么？→ 窗口动画谁驱动（WMS 编排，SF 呈现）？

Q5.2 addWindow 的审验漏斗?

【深度解答】四步从粗到细：屏（DisplayContent 收不收）→token（对得上注册的 WindowToken 吗，BadToken 的故乡）→类型权限（SYSTEM_ALERT_WINDOW 等逐类型）→建籍（WindowState+输入通道配对+层序指派）。顺序即安全语义漏斗，任何一步拒都有专有错误码回流。〔册5・一/二〕

【追问链】→ 对话框 BadToken 的时序解释（Activity attach 前弹）？→ 悬浮窗盖状态栏需要什么（type 层级+权限，非"画得高"）？→ 子窗口的 token 挂谁（父窗口）？

Q5.3 performTraversals 三幕的规矩与禁忌?

【深度解答】measure（自顶向下发 MeasureSpec，wrap_content 常两轮）→layout（拍板位置，期间再 requestLayout 即抛异常防震荡）→draw（软件逐条/硬件录 DisplayList 由 RenderThread 执行）。三幕只发生在 ViewRootImpl 一处；invalidate 是欠条（同帧合并），requestLayout 不触发 draw。〔册5・三〕

【追问链】→ onCreate 里 getWidth()==0 的原因（解析≠演算）？→ 为什么 UI 单线程（checkThread 免锁代价）？→ 硬件加速局部重绘为什么高效（子树重录整树重放）？

Q5.4 z 序三定律?

【深度解答】type 定车道（应用 1~99/子窗 1000+/系统 2000+）；Task 序定同车道名次（前台任务整体靠上）；动画期间暂存层例外（动画器接管结束归还）。z 序=layer（type 基线）+同层序+动画暂存，三句话解释一切"谁盖谁"问题。〔册5・一〕

【追问链】→ 输入法窗口的层位策略（perFocus 动态）；→ PopupWindow 为什么是子窗口（随父同进退）；→ 分屏的层位（Task 分区独立基线）？

Q5.5 Insets 是裁剪还是协商?

【深度解答】协商：InsetsController 与 WMS 按 InsetsType 分通道谈（status/nav/ime 独立），behavior 声明姿态；键盘动画两侧同帧同步（InsetsAnimationThread 免被主线程拖累）。adjustResize=Insets 全量应用重测；adjustPan=整窗平移不重测。键盘是独立窗口，只是协商出的量进入你的测量参数。〔册5・一〕

【追问链】→ edge-to-edge 的现代做法（WindowInsets API 自理）；→ 键盘"顶起布局"的测量链（insets→MeasureSpec）；→ 刘海屏（displayCutout）与 insets 的关系（另一通道）？

Q5.6 Choreographer 的两个实例与双时钟?

【深度解答】UI 线程 getInstance() 编排测量布局；RenderThread 的 getSfInstance()（ViewRootImpl.java:925 引用）服务硬件渲染提交——两个时钟域对应"主线程画什么、渲染线程怎么画"。App 的 VSync 与 SF 的 VSync 相位错开（生产/合成流水化）。〔册5・四〕

【追问链】→ 谁给 Choreographer 发 VSync（DisplayEventReceiver/BitTube，源自 SF 分发）；→ doFrame 晚到的补偿（跳帧计数对齐）；→ postCallback 与 postDelayed 的车道语义?

Q5.7 硬件加速的一帧后半程?

【深度解答】draw 录 DisplayList（RecordingCanvas）→RenderThread syncAndDrawFrame（Skia GPU 后端执行）→纹理/RenderNode 属性直接改不重录（translationX 流畅的秘密）→queueBuffer 入 BufferQueue（三缓冲）→SF 下个 VSync 合成（Overlay 直通优先）→present。掉帧责任由 FrameTimeline 四段标注（App/RenderThread/SF/Disp）。〔册5・八〕

【追问链】→ 为什么属性动画不 invalidate（改 RenderNode 矩阵）；→ Overlay 命中条件（不混合/无缩放/格式匹配）；→ 强制 GPU 渲染开关改变了什么（软件层也走 GPU）？

Q5.8 窗口不显示的排查三步?

【深度解答】①dumpsys window windows 查该窗：mViewVisibility/isReadyForDisplay/SurfaceControl 是否创建——停在 addWindow（无 relayout）还是首帧未提交；②查层位与 token（被盖/未挂树）；③perfetto 看 traversal 是否发生。三步分别对应"存在了吗/排对了吗/画了吗"。〔册5・附录〕

【追问链】→ Surface 拿到但黑屏（无内容提交，首帧未 queue）；→ 动画后窗口消失（暂存层归还失败/visibility 错）；→ 无障碍服务报告的"可显示"与实际不符（accessibility 视图与窗口树差异）？

Q5.9 多屏/折叠屏的窗口世界?

【深度解答】每屏一个 DisplayContent（窗口树/Insets/焦点独立 per-display）；Task 可跨屏迁移（拖拽分屏=Task 重组而非窗口搬运）；折叠态变化=配置变更广播（重建事务，册4 §2.2 同款）。焦点 per-display 但输入 dispatcher 全局只有一个 focusedWindow per display 的集合。〔册5・六〕

【追问链】→ 分屏两个 App 的输入怎么分（per-task 焦点+命中）；→ 外接屏默认显示谁（uniqueId/displayId 路由）；→ 投屏镜像（VirtualDisplay+镜像 Layer）？

Q5.10 ViewRootImpl 为什么是"窗口总管"?

【深度解答】它一身五职：View 树的根（assignParent 冒泡终点）、窗口 binder 通道（Session）、输入接收者（InputEventReceiver+InputStage）、帧编排者（Choreographer 客户）、Surface 持有者（生产者接口）。Activity/PhoneWindow/DecorView 只是结构，ViewRootImpl 才

把结构接入系统——所以 UI 的所有"与系统打交道"都经它。〔册5・二/三〕

【追问链】→ 一个进程几个 ViewRootImpl（每窗口一个，WindowManagerGlobal 登记）；→ checkThread 在哪类调用触发（traversal 族）；→ SurfaceView 为什么能独立刷新（独立 Surface 打洞，与窗口 Surface 并行提交）？

第六章 ROM / BSP / 刷机（12 题）

Q6.1 BSP 五件套是什么?

【深度解答】板级支持包=①内核+dtb/dtbo（本树 device/google/*-kernel 的 Image.lz4/dtbo.img）②vendor HAL 与其 rc/init 脚本 ③vendor sepolicy ④BoardConfig/Product 产品配置（device/oem/product）⑤固件（modem/tee/bluetooth…）。判断一个 ROM"支持一块新板"就是五件套齐且接口对齐 VINTF。〔notes・07/册1・三〕

【追问链】→ 板卡 first boot 卡在哪查（dtb 不匹配→kernel panic/找不到 console）；→ 新 HAL 该放哪个分区（vendor，接口在 hardware/interfaces）；→ sepolicy vendor 与 system 怎么协同（public/private 划分）？

Q6.2 device/ 目录在构建里的角色?

【深度解答】device/<oem>/<product> 是"产品身份证"：AndroidProducts.mk 列产品名、BoardConfig.mk 定板级（分区尺寸/架构/内核预编译路径）、device.mk 选模块与 overlays。Soong 由此展开变体（书1 §7.2）——lunch 的目标名就来自这里。本树含 amlogic/linaro/google 等多家参考。〔notes・07〕

【追问链】→ 新建产品的最小集（AndroidProducts+BoardConfig+device.mk）；→ PRODUCT_ 变量与 BOARD_ 变量的分工（产品 vs 板）；→ overlay（RRO）怎么改默认值不碰框架？

Q6.3 fastboot 与 fastbootd 的区别?

【深度解答】fastboot：bootloader 里的协议栈（system/core/fastboot/main.cpp:31），可刷 boot/system 等物理分区、解锁；fastbootd：userspace fastboot（走 adbd 通道），能刷动态分区（super 内的逻辑分区）——因为 bootloader 看不懂 super 里的 metadata，只有跑起来的系统（或 recovery）能。A/B 时代逻辑分区的刷机必经 fastbootd。〔notes・07〕

【追问链】→ fastboot reboot-bootloader 怎么实现（BCB/reboot reason）；→ 解锁会清 data 吗（会，防绕过加密）；→ 刷 GSI 的标准流程（解锁→fastboot init boot...→fastboot reboot fastboot→flash system）？

Q6.4 AVB 校验链与四色屏?

【深度解答】vbmeta 是根：内含各分区哈希（或 hashtree 描述符）+回滚索引+公钥；bootloader 逐级验签（boot→vbmeta→system…）。结果四色：绿（锁定全过）、黄（锁定但自定

义根）、橙（解锁）、红（校验失败拒启）。运行时 dm-verity 续验 system 只读分区。〔册1・七/notes・07〕

【追问链】→ avbtool 在构建里哪一步调用（build/make/core/Makefile:1054/4932）；→ 回滚索引防什么（降级攻击）；→ 锁定设备刷第三方 magisk 为何黄屏（boot 被改，自定义验签键路径）？

Q6.5 OTA 的一生（A/B 全流程）？

【深度解答】update_engine（system/update_engine）从源（OTA 包/serving）下载→校验 payload 签名→后台槽写入（未激活槽）→postinstall（动态分区 COW 快照若虚拟 A/B）→boot_control 标记新槽可启动→重启进新槽→成功后 slot 标 active、旧槽可复用；连续失败自动回退旧槽。Payload 的 manifest/metadata 与 delta（差分包）是核心数据结构。〔notes・07〕

【追问链】→ 虚拟 A/B 怎么省空间（COW，写时复制到 data）；→ A/B 的代价（存储双份 boot 分区）；→ 静默升级（seamless）的用户感知边界（重启那一刻）？

Q6.6 recovery 模式还剩什么职责?

【深度解答】A/B 后 recovery 大幅瘦身：sideload（adb 推包，bootable/recovery）、factory reset、部分厂商救援模式（进入内核小环境而非完整系统）。传统"卡刷"已非主线（A/B 无 recovery 分区刷写语义，或仅保留薄壳）。recovery_ui 与 minui 是它的 UI 栈。〔notes・07〕

【追问链】→ 进入 recovery 的路径（BCB boot-recovery/fastboot；）；→ sideload 与 update_engine 的关系（互斥的两种投递）；→ 双清清的是哪两层（data+cache，userdata 与回滚索引）？

Q6.7 刷机镜像家族速记?

【深度解答】boot（kernel[+GKI]）、init_boot（通用 ramdisk，13 起）、vendor_boot（vendor ramdisk）、dtb/dtbo（设备树与覆盖）、vbmeta（AVB 根）、system/vendor/product/odm/system_ext（只读主分区，super 动态切分）、userdata、super（动态分区容器）。产物规则在 build/make/core/Makefile（:846 dtb.img、:985 --dtb 参数）。〔notes・07〕

【追问链】→ GSI 刷哪个（system.img，CTS 认证设备必须可刷）；→ odm 与 vendor 的分工（板级 vs SoC 级）；→ super.img 怎么改子分区大小（lpunpack/lpmake 重建）？

Q6.8 新板卡 first bring-up 的五步?

【深度解答】①kernel 能起（dtb/console/rootfs）②init 进二阶段（分区表/挂载对）③关键服务活（servicemanager/SF/logd 的 rc 与 sepolicy）④显示与输入通（GPU/触摸 HAL+dtbo）⑤过 VINTF 自检（manifest/matrix 对齐）。每步都有"标志性日志"可查——bring-up 的方法论就是"用日志换进度"。〔册2 全册/notes・07〕

【追问链】→ 卡在 kernel 的常见因（dtb 不匹配/console 串口错）；→ 卡在 init 二阶段（fstab/avb/selinux 标签）；→ SF 起但黑屏（HAL/overlay/drm 少一层）？

Q6.9 ROM 定制的"正确姿势"四件套?

【深度解答】①预装应用（system/product 分区+privileged 白名单，册6 §五续）②默认行为（RRO overlay 改资源不动框架）③默认设置（SettingsProvider defaults/vendor 配置）④开机向导（SetupWizard 产品件）。原则：**能在 product/overlay 层解决就不碰 system/ 源码**——否则每轮 AOSP 合并都是灾难。〔册6/notes · 07〕

【追问链】→ 为什么不推荐直接改 framework 源码（升级合并冲突）；→ overlay 能改什么不能改什么（资源值可、代码逻辑不可）；→ 多产品共存的工程组织（device 树继承 lunch 链）?

Q6.10 dm-verity 与数据完整性?

【深度解答】只读分区（system/vendor…）挂载时建 hashtree（merkle），访问块时内核校验（第一块失败→重启/降级 per 模式）；vbmeta 存树根。检测到篡改（root 后改 system）即拒绝读取——配合 AVB 构成"启动前验根、运行中验块"。〔册1 · 七〕

【追问链】→ verity 与 AVB 的分工（启动信任根 vs 运行时块校验）；→ fec（前向纠错）干什么（小损坏自愈）；→ userdebug 为什么能关（disable-verity 调试豁免）?

Q6.11 Cuttlefish 是什么、为什么系统开发离不开它?

【深度解答】本树自带的虚拟设备（device/google/cuttlefish）：以用户态进程模拟手机（crosvm 虚机+虚拟传感器/触摸/显示），宿主机直接跑 AOSP 全栈，支持 adb/fastboot/streaming。价值：免真机做系统实验、CI 全量验证、GSI/内核矩阵测试。系统开发者的"本地沙箱"标准答案。〔notes · 07〕

【追问链】→ 它的 HAL 怎么虚拟（ranchu/cuttlefish HAL 实现）；→ 性能与真机差在哪（GPU virgl/gfxstream）；→ 与模拟器（ qemu arm）的区别（跑真 AOSP 产品构建）?

Q6.12 刷机/系统开发的常用命令拓扑?

【深度解答】四层记忆：宿主工具（fastboot flash boot boot.img / adb reboot bootloader|fastboot|recovery）；设备查询（getprop ro.* / dumpsys -l / service list）；系统操作（pm/svc/cmd activity|window|input|surfaceflinger）；观测（logcat -b events|crash、dumpsys window/input/package、perfetto）。每条命令对应前六册的一个子系统——命令即索引。〔各册附录〕

【追问链】→ cmd activity 与 am 的关系（同一服务的两个门）；→ 无 root 抓开机日志（bootchart/events log）；→ 串口日志与 logcat 的层次（console=dmesg vs 框架）?

第七章 综合场景题（8 题）

Q7.1 场景：用户点击图标 500ms 后才见首帧，怎么定位?

【深度解答】分段定界：am start -W 三段（WaitTime/TotalTime）+systrace：①孵化段（am_proc_start→attach）异常长？查 zygote 忙碌/USAP/进程池；②事务段（realStart→onCreate）

短则正常；③onCreate→首帧（inflate/traversal/GPU）业务大头。按三段归因，各自有专属轨道与锚点（册4 §八实战流程）。〔册4 · 八/册5 · 四〕

【追问链】→ 三段各占多少算正常（孵化 100-300ms/事务 10ms 级/首帧业务定）；→ 若 TotalTime 正常但用户仍觉慢（感知层：动画启动延迟/图标点击反馈）；→ 线上无 trace 怎么近似（埋点分段上报）？

Q7.2 场景：系统升级后某厂商 App 崩溃率暴涨，系统侧还是应用侧?

【深度解答】系统侧排查序：①平台行为变更（targetSdk 分水岭 M/Q/R 的权限与后台限制）②接口兼容（非 SDK 灰名单收紧，veridex 检测）③组件策略（FGS/通知/广播限流收紧）④时机变化（调度更激进致首启动竞态）。若崩溃栈在框架 API 边界但触发条件是"进程被杀重建"，多半是冻结/回收策略——系统"没错"，是契约边界移动了。〔册4 · 六/册6 · 四〕

【追问链】→ 如何复现（冻结窗口期+低内存模拟）；→ 应用侧正确的抗性设计（状态持久化/幂等重建）；→ 报告给 Google/厂商的正确姿势（bugreport+veridex 报告）？

Q7.3 场景：整机偶发输入全失灵 3 秒自愈?

【深度解答】层次定界：getevent 有流→内核正常；dumpsys input 的 dispatcher 队列暴涨→投递端堵（焦点窗口的 App 消费不动或 ANR 计时中）；队列正常但 App 无响应→通道/焦点瞬变（窗口切换动画期）。3 秒自愈指向"某窗口短暂僵死+超时回执/ANR 逃逸（dialog 出现）"——抓现场：连续 debuggerd -b 抓目标窗口栈。〔册5 · 五〕

【追问链】→ 为什么不是输入法劫持（Ime 截胡不阻塞其他窗口）；→ 全失灵与单 App 失灵的分界（focusedWindow 是否换人）；→ ANR 弹窗出现=输入恢复的机制（暂停投递+用户决策）？

Q7.4 场景：设计一个系统级"应用双开"?

【深度解答】三条路线对比：多用户（正统：PackageUserState per-user+userId 隔离，成本是 userId 资源与切换）；克隆应用（应用侧改包名多实例，破签名共享）；虚拟化容器（VirtAPP 类，framework hook，兼容性差）。系统工程师的正解是多用户基建+隐形 profile（工作资料机制复用）：AMS/PMS/WMS 天然 per-user，需要补的是 UI 聚合与通知路由。〔册6 · 五/册4 · 一〕

【追问链】→ 双开的存储隔离边界（per-user data 目录）；→ 通知如何区分（userId 附着 +SystemUI 分组）；→ 与厂商"应用分身"差异（多为方案一的白牌化）？

Q7.5 场景：车载/平板需要"分屏+多任务"，系统侧要动哪些层?

【深度解答】WMS 层：TaskDisplayArea 分区策略（WindowContainer 分派规则）；ATMS 层：分屏模式与 Task 重组（rootTask 复用）；输入层：per-task 焦点与命中；渲染层：双 Surface 独立管线；策略层：多窗口开关（config supportsMultiWindow）与厂商定制度量。AOSP 已有分屏基建（12+），产品化难点在交互策略而非框架能力。〔册5 · 一/六〕

【追问链】→ 两 App 同帧渲染的性能预算（GPU 与 SF 合成压力）；→ 拖拽分屏期间的窗口归属（Task 迁移动画）；→ 分屏下输入法给谁（焦点 task）？

Q7.6 场景：把 Android 13 移植到一块新开发板（bring-up 计划）?

【深度解答】六周节奏模板：W1 kernel/dtb 起 init；W2 分区+sepolicy 跑通二阶段；W3 显示输入通（SF/触摸）；W4 PMS 扫描全量包+开机到 Launcher；W5 VINTF/CTS 基线过；W6 性能与稳定性（bootchart、monkey）。每步验收物=日志证据；风险最大处=GKI 内核匹配与 HAL 版本（VINTF 对不齐全盘皆输）。〔notes · 07/册2〕

【追问链】→ 先 bring-up display 还是 touch（display，可 logcat 盲调）；→ VINTF 对不齐的现象（服务起不来/CTS fail 明细）；→ 何时引入厂商 BSP 包（W2 后，逐 HAL 合入）？

Q7.7 场景：面试官问"读系统源码你最大的收获？"

【深度解答】高分模板=具体锚点+方法论：例——"读 Binder 十跳后，我对'身份在内核采集'的安全模型有了肌肉记忆，之后审任何 IPC 方案先问凭证谁填；读 init 单线程模型后，我把'确定性优先于吞吐'用在了一个调度器设计上。"要点：**一个让你改变设计观的具体机制**，而不是"我读了多少行"。〔各册 · 设计决策章〕

【追问链】→ 这套源码观怎么迁移到非 Android 系统（分层/身份/事务化是通用的）；→ 六册里最推荐新人先读哪册（第一册+第二册：地图与主线）；→ 保持源码新鲜的习惯（release notes+Gerrit 关注几个核心仓库）？

Q7.8 场景：给你一台卡顿设备做 30 分钟体检?

【深度解答】五步流水：①定界：dumpsys gfxinfo/framestats 或 perfetto 抓 10s（掉帧责任段）；②系统水位：dumpsys meminfo+lmkd 日志（是否内存挤压杀进程循环）；③调度：top/dumpsys activity processes（adj 错乱/缓存风暴）；④单点深挖：FrameTimeline 红帧下钻对应册 · 章（App traversal? SF 合成？）；⑤复验：改一项再测。30 分钟的本质是"用流水线代替直觉"。〔册5 · 四/八、册4 · 六〕

【追问链】→ 优先怀疑系统还是应用（数据说话：FrameTimeline 分责）；→ 内存压力型卡顿的特征（am_kill+am_proc_start 频繁成对）；→ 给用户的一句话结论怎么写（现象+责任段+建议三段式）？

第八章 深水区题（20 题）

Q8.1 ART 的并发复制 GC 为什么能做到毫秒级停顿?

【深度解答】三件套：region 化堆（RegionSpace，搬移以区域为单位）+ **读屏障**（引用读取时顺带检查 forwarding 指针、按需搬移——搬移与应用并发）+ 卡表/记忆集（跨代引用脏页索引，避免全堆扫描）。STW 只剩根初始标记与最终修正两个小阶段。代价：每条引用读指令多几条屏障指令——用 CPU 换延迟。锚点 art/runtime/gc/collector/concurrent_copying.cc。〔册4 · 七〕

【追问链】→ LOS 大对象为什么独立空间? → GC 与 zygote COW 的互动（fork 前 gcAndFinalize 的意义）？ → 观测 GC 的三件工具（meminfo/systrace gc 轨/heapdump）？

Q8.2 MTERP 为什么用汇编写解释器?

【深度解答】C 解释器每条字节码都有函数调用/栈帧开销；MTERP（art/runtime/interpreter/mterp/arm64ng）把"取指-分发-执行"编成一段手写汇编：分发表跳转、虚拟寄存器钉在物理寄存器、线程切换点内联——吞吐近翻倍。它不是 JIT：仍是一条条解释，只是"解释的地板"被垫高。〔册4・七〕

【追问链】→ MTERP 与 quickened 字节码的关系？→ 为什么不全部 JIT（编译也有成本，冷代码白编）？→ profile 如何决定谁进 JIT 谁进 AOT?

Q8.3 lmkd 的 PSI 事件是什么、比轮询好在哪?

【深度解答】PSI（Pressure Stall Information）是内核度量的"因内存压力而停顿的任务时间占比"；lmkd（lmkd.cpp:3735）订阅 some/full 两档阈值事件，压力到达即刻按 adj 位图选杀——**事件驱动、零轮询延迟**，且能区分"有富余但抖动"与"真饥荒"。旧内核态 LMK 只看剩余水位，粒度粗且策略不可更新。〔册4・七〕

【追问链】→ LMK_PROCPRIO 谁在发（AMS OomAdjuster）？→ zram writeback 的触发条件？→ 为什么杀了还卡（swap thrash， PSI full 档兜底）？

Q8.4 冻结（freezer）与杀（kill）的配合顺序?

【深度解答】CachedAppOptimizer 先冻结（cgroup v2 freezer，CPU 全停+BR_FROZEN_REPLY 快速失败+广播 deferred），内存仍紧才升级 kill——冻结是可逆档（解冻省一次冷启动），kill 是不可逆档。观测点：dumpsys activity 里 cachedFreezer 状态与 logcat 的 freeze/unfreeze 事件。〔册4・七/册3・十〕

【追问链】→ 冻结进程的 binder 调用方如何兜底（重试/退避）？→ 解冻窗口谁开（AMS 在调度组件前）？→ 冻结与 JobScheduler 的互斥怎么处理?

Q8.5 cgroups.json 与 task_profiles.json 的分工?

【深度解答】cgroups.json 定义**层级挂载**（哪些控制器、挂哪、属主）；task_profiles.json 定义**命名任务档**（如 ProcessCapacityHigh/SchedUgTop=一组 cgroup 写入+调度参数），服务 rc 与 framework 代码引用档名而非底层路径——**单一事实源+可移植**（Android 版本/厂商差异由 profiles 吸收）。〔册4・七〕

【追问链】→ 新建一个自定义档的步骤？→ cpuset 与 uclamp 各管什么（绑核 vs 提升裕量）？→ init.rc 的 task_profiles 行何时兑现（Service::Start）？

Q8.6 传感器为什么用 FMQ 而不是 Binder?

【深度解答】传感器事件高频小包（200Hz+ 的 9 轴流），binder 每次事务有组包/调度成本；AIDL HAL 的 EventQueue（ISensors.aidl:129）用 **FMQ（共享内存环形队列）+ 事件旗标**：写者（HAL）入队、读者（SensorService）批量收割，控制面（启停/配置）才走 binder——数据面/控制面分离的传感器版。〔册5・八〕

【追问链】→ direct channel 比 FMQ 更快在哪（连 SensorService 都绕过，DSP 直写共享内存）？→ 批处理（batch）的 FIFO 硬件前提？→ 融合传感器（CorrectedGyro 等）在哪算（service 侧

Fusion.cpp）？

Q8.7 电源键在哪儿被"没收"的？

【深度解答】两道闸：入队前 interceptKeyBeforeQueueing（InputDispatcher.cpp:3935→InputManagerService.java:3003→WMS InputPolicy）与分发前 interceptKeyBeforeDispatching；电源/音量组合（截图、复位）在系统策略层直接消费并标记"不再分发"——App 永远收不到原始事件。这也是防键盘记录类攻击的机制层。〔册5・八〕

【追问链】→ App 能拦截音量键吗（能，除非系统已消费）？ → 按键与触摸的"没收权"差异？ → 无障碍为什么能读到被拦按键（a11y 通道在拦截上游）？

Q8.8 预测性返回改变了什么？

【深度解答】旧模型：返回=KEYCODE_BACK 到达才响应，用户无预览；新模型（OnBackInvokedDispatcher.java:28）：手势按下即向目标窗口发渐进回调（progress/start/cancel），App 可实时预览并取消，系统侧同播手势动画——**从"事件通知"进化为"过程协商"**。迁移期两套并存，按 targetSdk/opt-in 切换。〔册5・八〕

【追问链】→ 与手势导航的手势动画如何协同？ → WebView 内部返回谁优先？ → 未适配 App 的降级行为（自动回退旧 KeyEvent）？

Q8.9 IME 卡顿常见根源？

【深度解答】文字协议滥用：InputConnection 每次查询都是一次 binder 往返（getTextBeforeCursor/setComposingText…），输入法每键多次查询即雪崩；IMMS 侧重绑（InputMethodBindingController）与焦点切换竞态也会表现为"键盘冻住"。排查：systrace 的 IME 轨+文本协议调用计数；优化：批量编辑（BatchEdit）、减少上下文查询。〔册5・八〕

【追问链】→ 键盘弹出的 Insets 协商链？ → 多输入法切换的绑定生命周期？ → 手写就地输入（13）绕开了什么（不走文本框、Surface 直接递笔迹）？

Q8.10 无障碍服务为什么被严格管控？

【深度解答】a11y 服务可读全屏内容+合成触摸/按键（AccessibilityManagerService 的事件流与 GestureDescription 注入）——等同"上帝权限"。13 起强制安装来源受限+显式授权弹窗、限制后台绑定。设计启示：能力越普惠（残障辅助）越要防能力滥用（羊毛/监控）。〔册5・八〕

【追问链】→ AccessibilityNodeInfo 快照的生成成本（View 树按需序列化）？ → 触控朗读与 TalkBack 的事件消费差异？ → 自动化测试（UiAutomator）用的是不是同一条通道（是）？

Q8.11 neverallow 在 SELinux 里是什么地位？

【深度解答】策略"宪法条款"：编译期强制检查，任何 allow 与既有 neverallow（如 system/sepolicy/private/domain.te:119/:137 起的数百条）冲突即构建失败——**把"永远禁止"从 review 纪律升级为机器保证**。写策略的正确姿势：先查 neverallow 是否已封死此路，再谈 allow。〔册1・十六〕

【追问链】→ audit2allow 为什么不能无脑用？→ seapp_contexts 给谁打标（App 进程域，按包名/uid）？→ MLS 类别（s0）在实际策略里怎么用？

Q8.12 APEX 回滚的两次启动判定是什么？

【深度解答】staged 会话激活后处于"试运行"：boot 阶段 apexd/_bootflag 标记，若本会话期间系统连续两次未正常 boot 到 reckoned 状态，下次启动自动回退旧版 APEX 并禁新会话冷却——与会话状态机（staged→activated→reckoned）一体。这是 Mainline 敢高频发版的"保命绳"。〔册 1・十六〕

【追问链】→ 回滚与 APK 的 rollback（RollbackManager）什么关系（不同子系统、同一思想）？→ 哪些 APEX 不允许回滚（与 boot 链绑定的）？→ 如何观测（apexd --list/dm 命令族）？

Q8.13 链接器命名空间解决了什么 Treble 痛点？

【深度解答】动态符号解析的隔离：framework 进程看不到 vendor 命名空间私有导出、vendor 进程用的是 vndbinder/libc 的 vendor 变体——**同名 so 双版本并存不互踩**（system/linkerconfig 生成各进程 linker config）。它发生在链接期，比运行期任何隔离都早。〔册 1・十七〕

【追问链】→ "dlopen failed: library not found"的命名空间排查法（readlink /proc/<pid>/maps + linker config 对比）？→ sphal/llndk/default 四类命名空间各放什么？→ NDK app 链的是哪个域（default，白名单 public libs）？

Q8.14 fastboot 为什么要有 userspace 版（fastbootd）？

【深度解答】动态分区（super）的元数据只有完整系统里的 liblp/dm-linear 能解析；bootloader 无此栈。逻辑分区的 flash 因此必须走 fastbootd（userspace，经 adbd 通道）；物理分区（boot/vbmeta…）才由 bootloader 版直刷。刷 GSI 流程"reboot fastboot"就是为了进入这个 userspace 环境。〔册2・十一〕

【追问链】→ A/B 槽后缀与 slot 切换谁管（boot_control HAL）？→ getvar:max-download-size 决定什么（分块下载尺寸）？→ 解锁状态影响哪些校验（bootloader 层 AVB，dm-verity 运行层仍提示）？

Q8.15 update_engine 的 Action 流水线？

【深度解答】InstallPlan→FilesystemVerifier（验旧）→Download/DeltaPerformer（增量块搬运+bz4→写后台槽）→Postinstall→boot_control 切槽。任一 Action 失败整单回退、旧槽无损；payload 签名与哈希树两级校验。观测：logcat -s update_engine 与 `update_engine_client --status`。〔册2・十一〕

【追问链】→ 增量包怎么对齐新旧块（manifest 的 install operations）？→ 虚拟 A/B 的 COW 写在哪（/data，牺牲空间换速度）？→ postinstall 脚本能做什么（OTA 后 dexopt 触发）？

Q8.16 GKI 把什么从 boot 分区拿走了？

【深度解答】通用内核镜像：boot 只剩 Google 签名内核；厂商差异（模块/vendor ramstock→vendor_boot、dtbo、bootconfig 参数）外迁，内核与 vendor 由符号清单（KMI）锁界面——内核碎片化治理。bootconfig 是 cmdline 的结构化继任（androidboot.* 家族迁入）。〔册2・十一〕

【追问链】→ KMI 符号清单怎么维护（abi_gki_*.xml 白名单）？→ vendor_dlkm 模块谁加载（内核 module loader+init rc）？→ GKI 与 Project Treble 的关系（镜像版同一思想）？

Q8.17 installd 的缓存清退什么时候触发?

【深度解答】磁盘水位紧张时系统存储策略（StorageManagerService）调用 installd 的 rm_cache 系列：CacheTracker 按 app 聚合 cache 用量、先清"最久未用+无活跃进程"的份额，必要时清至配额；App 无感知（cache 语义本就允许蒸发）。排查"cache 变小/quota 异常"第一站 dumpsys diskstats + installd 日志。〔册6・八〕

【追问链】→ cache 与 data 的物理区分（目录位置+清退权差异）？→ 配额的底层实现（fs quota 或 xattr）？→ CrateManager 的 crate 是什么（scoped storage 的分箱目录）？

Q8.18 Auto Backup 与 Key/Value Backup 的边界?

【深度解答】Auto Backup：全量 tar 流（adb backup 同源），25MB 上限，应用零代码；K/V：BackupAgent 逐键增量、需写代码，适合状态型数据。共同基建：BMS（services/backup）调度、传输层可插拔（Google/OEM）、恢复发生在安装流水线"校验后首启前"窗口。工作资料独立密钥域。〔册6・八〕

【追问链】→ backup 过期标记谁打（安装会话后，BMS）？→ 恢复失败会阻断首启吗（不阻断，只标记待重试）？→ D2D 迁移走哪条通道（近场直传，不走云）？

Q8.19 split APK 的加载与一致性?

【深度解答】系统侧一个 Package 多 APK：安装会话一次提交（签名全一致）；运行时 LoadedApk 建 split classloader 路由资源与 dex（base 与 config.split_xx 按需组合）；Feature Delivery 的按需安装只是会话化在模块粒度的复用。风险点：split 间资源覆盖顺序、跨 split 反射（同签名同 visibility 才稳）。〔册6・八/册4・一〕

【追问链】→ config split（mcc/mnc/abi/density）谁挑（最佳匹配算法在 LoadedApk/AssetManager）？→ 按需模块卸载的边界（保留 base 依赖闭包）？→ Play Install 与 pm install-session 的关系（同一会话 API）？

Q8.20 RPC backend 会取代 Binder 吗?

【深度解答】13 树里 RpcState.cpp 标志的 binder_rpc 实验分支：Binder 协议跑在 Unix socket/TLS 等任意传输上，面向多设备（手机-车机-穿戴）——**Binder 正在从"设备内 IPC"升维为"跨设备 RPC"**。但设备内主线不变：/dev/binder 的驱动优势（一次拷贝/内核身份）无法被用户态传输复制。答案是共存：域内 Binder、域间 RPC。〔册3・十二〕

【追问链】→ RPC 后端如何做安全（TLS+会话密钥，替代驱动身份）？→ 与跨设备"超级终端"框架的关系？→ AIDL 四后端（java/cpp/ndk/rust）在 RPC 下的复用？

第九章 补全题（10 题）

Q9.1 主线程死循环为什么"不卡"?

【深度解答】"卡"的前提是循环在忙转；而 Looper.loop（Looper.java:260）的 next() 无消息时经 MessageQueue.nativePollOnce（:72/:335）进入 native：NativeMessageQueue（android_os_MessageQueue.cpp:41/107）→ libutils Looper.pollOnce（Looper.cpp:181）→ **epoll_wait（:245）睡眠**，线程挂起零 CPU；nativeWake（MQ:73/:443）写管道唤醒。主线程=一台"用 epoll 武装的优先级队列"，idle 成本近乎为零。〔册4 · 八〕

【追问链】→ 同步屏障如何插队（异步消息优先，书5 §3.1）？→ epoll 还监听谁（输入 socket/IdleHandler fd——书5 输入回调的底座）？→ ANR 时这台发动机怎么了（主线程在处理某条消息/睡眠但队列被屏障压住）？

Q9.2 post(Runnable) 和 sendMessage 的区别?

【深度解答】殊途同归：post 把 Runnable 装进 message.callback，dispatchMessage（Handler.java:97）第一优先级先跑 callback，其次 mCallback，最后 handleMessage——**API 差异只是回调挂载点不同**。时序上都要走同一队列同一 when 排序；跨线程 post 即"投递到目标 Looper"的标准姿势（binder 线程转主线程即此，书4 §C.3）。〔册4 · 八〕

【追问链】→ handleMessage 里再 sendMessage 会怎样（正常入队，潜在自旋需警惕）？→ IdleHandler 何时跑（next() 队列空的空档）？→ 消息池复用（obtain/splice 回链表）为什么重要（免 GC 抖动）？

Q9.3 JNI 三类引用各管什么?

【深度解答】局部引用：方法帧内生命周期（循环里长期持有需显式 DeleteLocalRef——表有上限）；全局引用：跨线程/长存（NewGlobalRef/DeleteGlobalRef 成对，BinderProxy 的 nativeData 即现场）；弱全局：不保活（promote 失败即对象已亡，JavaBBinderHolder 缓存即现场）。异常规则：JNI 异常不清空、native 继续跑，必须 ExceptionCheck 早退。〔册1 · 十八〕

【追问链】→ 静态注册 vs 动态注册（RegisterNatives vs JNI_OnLoad）？→ 框架为什么全静态（启动一次性注册大表，可校验签名）？→ native 线程怎么 attach（AttachCurrentThread，detach 泄漏线程局部表）？

Q9.4 为什么 Application Context 启动 Activity 会报错?

【深度解答】隐式约束：startActivity 需要窗口令牌（token），而令牌来自 Activity/窗口体系——Application 的 ContextImpl（ContextImpl.java:192）没有绑定窗口。例外：加 FLAG_ACTIVITY_NEW_TASK 可以（走任务栈而非"在调用者窗口上叠"路径）。本质是 Context 族谱的权限投影：**Context 是"环境句柄"，能干什么取决于它背后的实体**。〔册1 · 十八〕

【追问链】→ Service 的 Context 有令牌吗（没有 UI 令牌，同限制）？→ 系统侧"系统 Context"（createSystemContext）从哪来？→ getSystemService 的缓存挂在谁身上（ContextImpl，per-context）？

Q9.5 sp/wp 与 Binder 引用计数命令什么关系？

【深度解答】sp<T>/wp<T>（RefBase.h:281）是进程内的强/弱计数；BC_ACQUIRE/BC_INCREFS（书3 §5.4）把它**跨进程镜像**：本端 sp 构造→发 ACQUIRE→驱动在对端建引用记账，两端各自确定性析构。所以 Binder 的"对象生命周期"= RefBase 本地 + 驱动远程账本的对偶；wp.promote 失败≈对端已死（死亡通知兜底）。〔册1・十八/册3・三〕

【追问链】→ 为什么不用 shared_ptr（先于标准、弱语义可裁剪）？→ 悬垂 wp 的 promote 语义？→ 死亡通知与 ref 归零的先后（ref 尽→node 销毁→BR_DEAD_BINDER）？

Q9.6 PendingIntent 的"代执行"语义？

【深度解答】它打包"意图+创建者身份+权限"成 IIntentSender 凭据（PendingIntentRecord.java:54 存在 AMS）；持有者（通知/闹钟/Widget）调用 send 时，**以创建者身份执行**（除非 FLAG_NO_CREATE/代发限定）。安全三问：可取消（cancel 后 send 报 Canceled）；可变性（13 默认 FLAG_IMMUTABLE，防持有者篡改 intent）；越权（转交即让渡身份——只交给可信方）。〔册4・九〕

【追问链】→ 为什么通知点击不走普通 Intent（通知由 SystemUI 进程发出，无权冒充你）？→ immutable 的例外修改面（setActivityStack? 13 只允许 bundle 更新器显式放行）？→ AlarmManager 的 PendingIntent 与 App 重启（凭据在 AMS，重启后失效需重注）？

Q9.7 onActivityResult 的结果什么时候回来？

【深度解答】目标 setResult+finish → AMS 记账 → 发起方重新可见（resume）时以 **ActivityResultItem**（与 LaunchActivityItem 同族事务）下发 → handleSendResult → onActivityResult——**结果到达时机=发起方回前台**，中间隔着整个生命周期流转。这是"事务机器"复用的又一现场：回传不是新机制，是 ClientTransaction 家族多了一个 Item。〔册4・九〕

【追问链】→ 多层启动链的结果逐级回传顺序？→ activityResultRegistry（Jetpack）解决什么（分层封装 requestCode）？→ 单结果 API（ActivityResultContract）之下还是它吗（是）？

Q9.8 动态注册的广播为什么在主线程回调？

【深度解答】LoadedApk 为每个"Context+Receiver"建 ReceiverDispatcher（LoadedApk.java:107 内部类）：IIntentReceiver 的 binder stub 收到 BroadcastQueue 分发（binder 线程）→ 立即 post 到绑定的 Handler（默认主线程）→ onReceive。**dispatcher 是 binder 世界到 Handler 世界的桥**，同族的 ServiceDispatcher 管绑定。推论：onReceive 主线程别重活（goAsync 书4 §4.2）；泄漏根源=未 unregister 导致 dispatcher 与 binder 强引用链常驻。〔册4・九〕

【追问链】→ goAsync 的 PendingResult 在哪个线程计时（超时线不变）？→ LocalBroadcast 为什么不走这套（进程内直接分发）？→ manifest 接收者有 dispatcher 吗（无——AMS 直接经 ApplicationThread 分发）？

Q9.9 视频为什么用 SurfaceView 而不是普通 View?

【深度解答】SurfaceView 拥有独立 Surface（SurfaceView.java:239 构造即申请，updateSurface :154-160 独立控制流）：视频解码线程直接上屏，帧率与 UI 滚动解耦、不占 UI 帧预算；且常走 Overlay 直通（书5 §8）省一次合成。普通 View 必须并入窗口 Surface，跟随 Choreographer 节拍——高帧率流会被 UI 帧率钳制。TextureView 折中：可动画可变换但多一次 GPU 拷贝。〔册5・九〕

【追问链】→ "打洞"（setZOrderedOnTop=false）与洞上盖 UI 的合成代价？→ BLASTBufferQueue（12+）改变了什么（提交流与事务对齐）？→ 相机预览选哪个（SurfaceView，低延迟直通）？

Q9.10 属性动画比 View 动画"真"在哪?

【深度解答】View 动画只改 draw 矩阵（视觉骗术，位置与响应区不动）；属性动画经 AnimationHandler（AnimationHandler.java:24/:83）挂 Choreographer 的 **CALLBACK_ANIMATION** 车道，每个 vsync 插值→**改真实属性**→invalidate→TRAVERSAL 车道重测布局。车道顺序（动画先于遍历，书5 §4.1）保证同帧闭环。更快的变体：RenderThread 动画（RenderNode 属性直改不重录，书5 §8）。〔册5・九〕

【追问链】→ 插值器/估值器的扩展点？→ 为什么动画中频繁 GC 会掉帧（每帧对象分配→池化 ValueAnimator）？→ Choreographer 车道与 doFrame 补偿的协作（书5 §4.1）？

附录 面试策略五条

- **追问链是主考法**：每个答案预留两个"下钻点"，主动说出"这背后还有一个细节是……"引导到你熟的领域。
- **锚点背 20 个**：十跳、九阶段、三批服务、15+1/31、1MB-2页、5s/10s/20s、：824/:763/:3546……锚点脱口而出即"真读过"。
- **画图优先**：启动时序、Binder 十跳、窗口树、输入链路——白板画图比叙述快且可信。
- **知边界**：答"驱动源码不在本树、由 UAPI 反推"这种诚实声明，比编造内核细节加分。
- **把六册当弹药库**：本册每题的〔册・章〕即回查坐标——面试前夜重读各册设计决策章即可焕新。